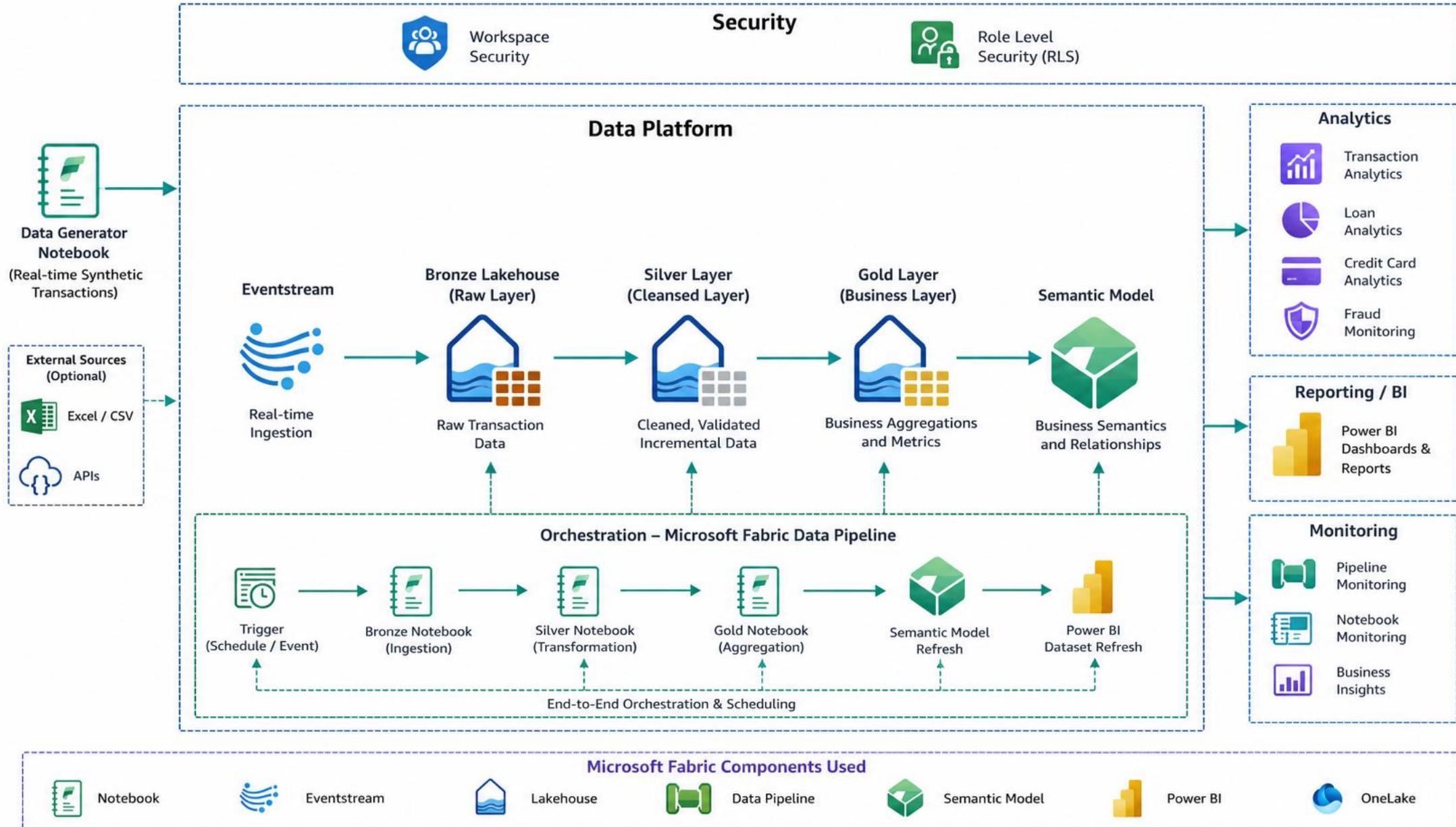


Smart Banking Analytics Platform

- Architecture



- **Data generator notebook**

Purpose: Generate synthetic banking transaction data in real time using Python (PySpark) within a Microsoft Fabric Notebook.

```
105     "Date": txn_date,
106     "TransactionType": txn_type,
107     "Amount": amount,
108     "Channel": channel,
109     "EventProcessedUtcTime": event_processed_time.isoformat(),
110     "PartitionId": partition_id,
111     "EventEnqueuedUtcTime": event_enqueued_time.isoformat()
112 }
113
114 batch = producer.create_batch()
115 batch.add(EventData(json.dumps(event_data)))
116 producer.send_batch(batch)
117
118 print(
119     f"Sent -> "
120     f"TransactionID={transaction_id}, "
121     f"CustomerID={customer_id}, "
122     f"Amount={amount}, "
123     f"Type={txn_type}"
124 )
125
126 time.sleep(2)
```

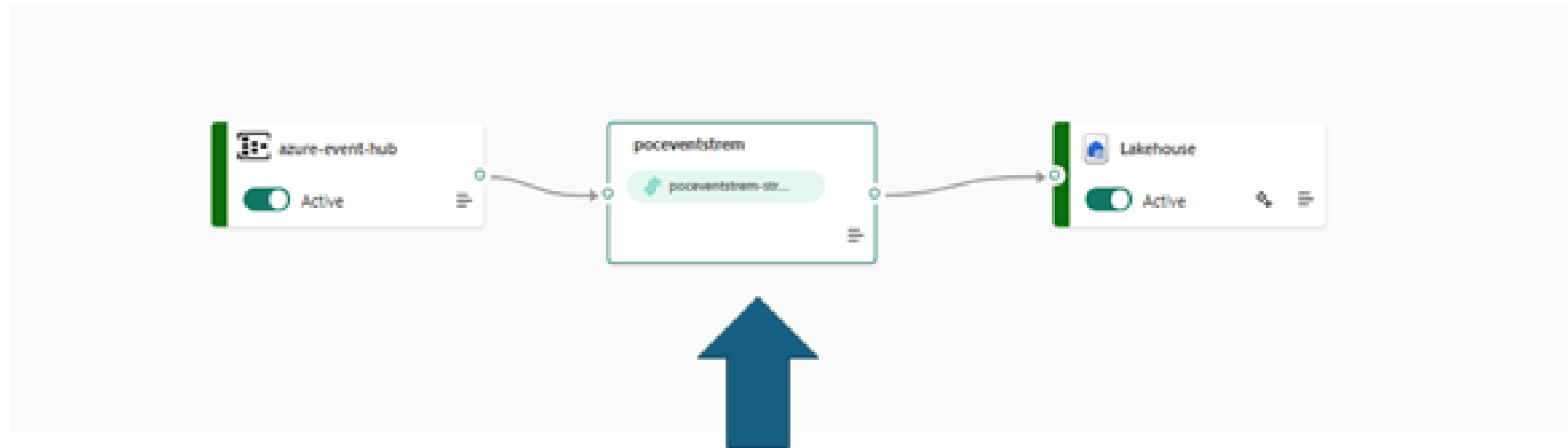
[*] * 56 min 13 sec - Session ready in 401 ms. Running PySpark (Python)

> Spark jobs (20 of 20 succeeded) Resources Log

... Sent -> TransactionID=831628, CustomerID=671, Amount=25866.83, Type=Withdrawal
Sent -> TransactionID=896387, CustomerID=164, Amount=16659.99, Type=Bill Payment
Sent -> TransactionID=129029, CustomerID=199, Amount=14187.85, Type=Deposit
Sent -> TransactionID=695438, CustomerID=295, Amount=27417.0, Type=Deposit
Sent -> TransactionID=708476, CustomerID=547, Amount=23661.79, Type=Bill Payment
Sent -> TransactionID=687698, CustomerID=826, Amount=31643.72, Type=Bill Payment
Sent -> TransactionID=295964, CustomerID=613, Amount=29000.65, Type=Transfer
Sent -> TransactionID=716613, CustomerID=412, Amount=3832.69, Type=Bill Payment
Sent -> TransactionID=745737, CustomerID=969, Amount=18714.29, Type=Withdrawal
Sent -> TransactionID=992289, CustomerID=339, Amount=28345.66, Type=Transfer
Sent -> TransactionID=141147, CustomerID=818, Amount=5156.96, Type=Withdrawal
Sent -> TransactionID=194668, CustomerID=921, Amount=28337.76, Type=Transfer
Sent -> TransactionID=927369, CustomerID=832, Amount=46540.21, Type=Transfer
Sent -> TransactionID=714782, CustomerID=839, Amount=25565.7, Type=Transfer
Sent -> TransactionID=468428, CustomerID=48, Amount=48375.32, Type=Bill Payment
Sent -> TransactionID=224327, CustomerID=492, Amount=42486.94, Type=Deposit
Sent -> TransactionID=176543, CustomerID=293, Amount=16797.55, Type=Bill Payment
Sent -> TransactionID=603814, CustomerID=722, Amount=15571.52, Type=Deposit
Sent -> TransactionID=605394, CustomerID=217, Amount=17018.12, Type=Bill Payment
Sent -> TransactionID=251669, CustomerID=886, Amount=21223.31, Type=Withdrawal
Sent -> TransactionID=247942, CustomerID=455, Amount=7394.46, Type=Bill Payment

- **Eventstream and Bronze Layer Ingestion**

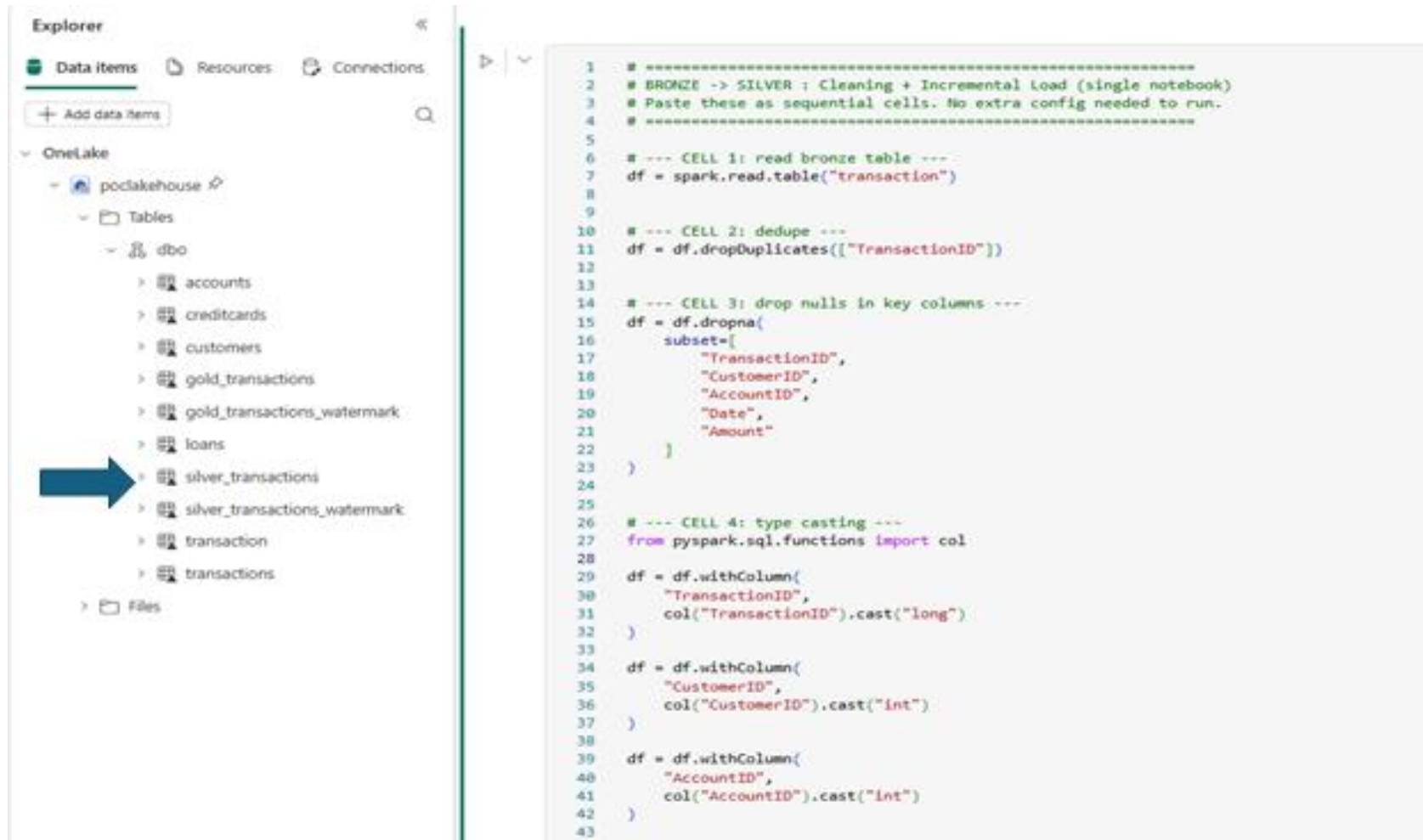
Purpose: Ingest real-time transaction data and store it as raw data in the Lakehouse Bronze layer.



- ▼  poclakehouse 
 - ▼  Tables
 - ▼  dbo
 - >  accounts
 - >  creditcards
 - >  customers
 - >  gold_transactions
 - >  gold_transactions_watermark
 - >  loans
 - >  silver_transactions
 - >  silver_transactions_watermark
 - >  transaction 
 - ▼  Files

- **Silver Layer – Data Cleansing and Incremental Processing**

Purpose: Transform raw bronze data into clean, validated, and analytics-ready data.

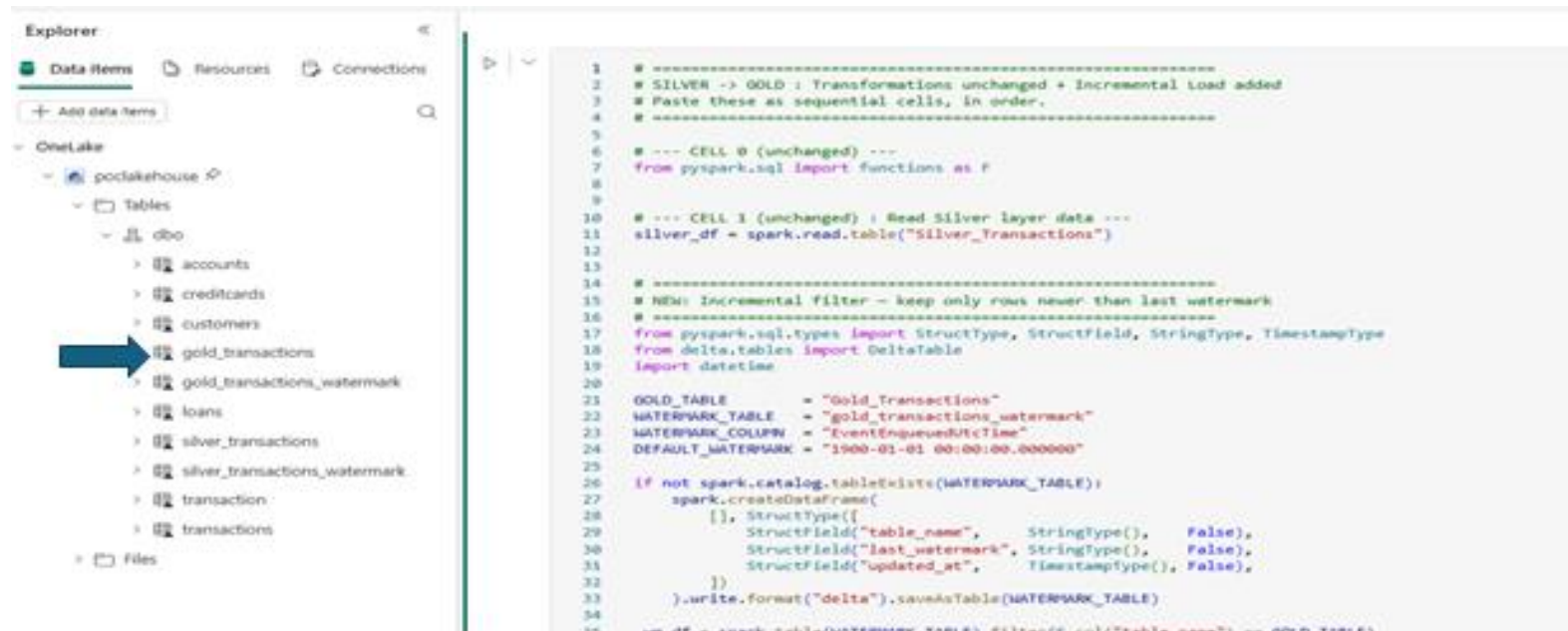


The image displays a data catalog interface on the left and a PySpark notebook on the right. In the catalog, the 'silver_transactions' table is highlighted with a blue arrow. The notebook contains a PySpark script that reads the 'transaction' table from the bronze layer and applies four cleaning steps: reading the table, deduping, dropping nulls in key columns, and casting data types.

```
1 # =====  
2 # BRONZE -> SILVER : Cleaning + Incremental load (single notebook)  
3 # Paste these as sequential cells. No extra config needed to run.  
4 # =====  
5  
6 # --- CELL 1: read bronze table ---  
7 df = spark.read.table("transaction")  
8  
9  
10 # --- CELL 2: dedupe ---  
11 df = df.dropDuplicates(["TransactionID"])  
12  
13  
14 # --- CELL 3: drop nulls in key columns ---  
15 df = df.dropna(  
16     subset=[  
17         "TransactionID",  
18         "CustomerID",  
19         "AccountID",  
20         "Date",  
21         "Amount"  
22     ]  
23 )  
24  
25  
26 # --- CELL 4: type casting ---  
27 from pyspark.sql.functions import col  
28  
29 df = df.withColumn(  
30     "TransactionID",  
31     col("TransactionID").cast("long")  
32 )  
33  
34 df = df.withColumn(  
35     "CustomerID",  
36     col("CustomerID").cast("int")  
37 )  
38  
39 df = df.withColumn(  
40     "AccountID",  
41     col("AccountID").cast("int")  
42 )  
43
```

- **Gold Layer – Business Aggregation and Analytics**

Purpose: Transform the cleansed silver data into business-ready datasets optimized for reporting and analytics.

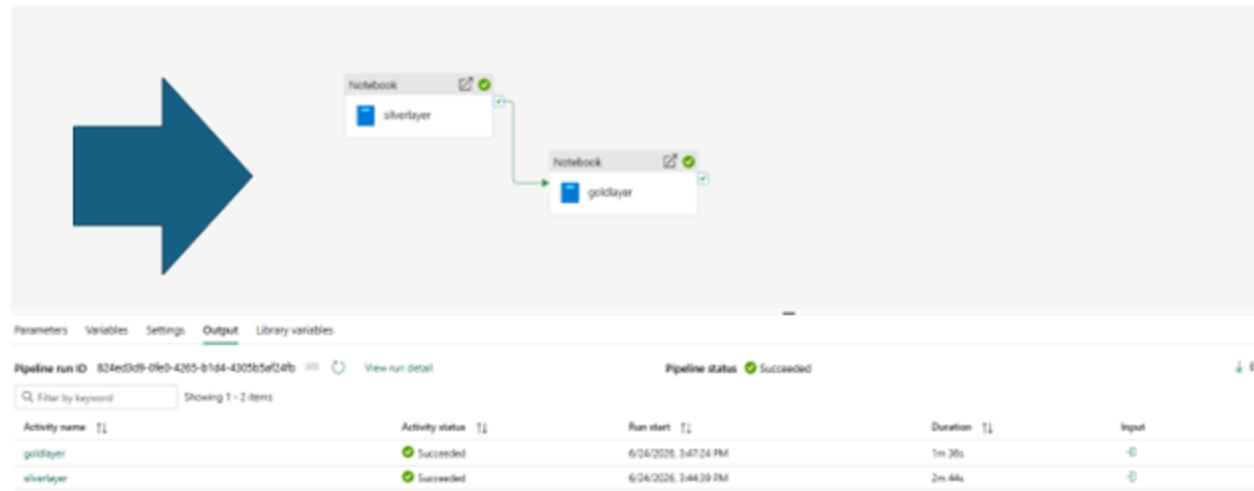


The screenshot displays the Databricks interface. On the left, the 'Explorer' sidebar shows a tree view of data items. Under 'Data Items', 'OneLake' is expanded, showing 'poclakehouse' and 'Tables'. Under 'Tables', 'dbo' is expanded, listing various tables. A blue arrow points to the 'gold_transactions' table. On the right, the code editor shows a PySpark script for transforming silver data into gold. The script includes comments for each cell, imports for PySpark SQL functions and DeltaTable, and code to read from 'Silver_Transactions', create a watermark table, and write the result as a Delta table.

```
1 # -----  
2 # SILVER -> GOLD : Transformations unchanged + Incremental load added  
3 # Paste these as sequential cells, in order.  
4 # -----  
5  
6 # --- CELL 0 (unchanged) ---  
7 from pyspark.sql import functions as F  
8  
9  
10 # --- CELL 1 (unchanged) : Read Silver layer data ---  
11 silver_df = spark.read.table("Silver_Transactions")  
12  
13  
14 # -----  
15 # NEW: Incremental filter - keep only rows newer than last watermark  
16 # -----  
17 from pyspark.sql.types import StructType, StructField, StringType, TimestampType  
18 from delta.tables import DeltaTable  
19 import datetime  
20  
21 GOLD_TABLE = "Gold_Transactions"  
22 WATERMARK_TABLE = "gold_transactions_watermark"  
23 WATERMARK_COLUMN = "EventEnqueuedUtcTime"  
24 DEFAULT_WATERMARK = "1900-01-01 00:00:00.000000"  
25  
26 if not spark.catalog.tableExists(WATERMARK_TABLE):  
27     spark.createDataFrame(  
28         [], StructType([  
29             StructField("table_name", StringType(), False),  
30             StructField("last_watermark", StringType(), False),  
31             StructField("updated_at", TimestampType(), False),  
32         ])  
33     ).write.format("delta").saveAsTable(WATERMARK_TABLE)  
34  
35 silver_df = spark.table(WATERMARK_TABLE).filter(F.col("table_name") == GOLD_TABLE)
```

- **Data Pipeline – Orchestration and Scheduling**

Purpose: Automate the execution of data processing workflows and ensure data is refreshed on a scheduled basis.



- **Data Pipeline – Orchestration, Scheduling, and Notebook Refresh**

Purpose: Automate the execution of data processing workflows and schedule notebook refreshes to keep analytics data up to date.

Schedules



Every minute

 On

Last successful update	Next update	Job type
06/19/2026, 4:12 PM	less than a minute	Pipeline

Show more

Time zone: (UTC+05:30) Chennai, Kolkata, Mumbai, New Delhi

ID: dbb29905-72ee-4f2a-82bf-b6a2394f99df

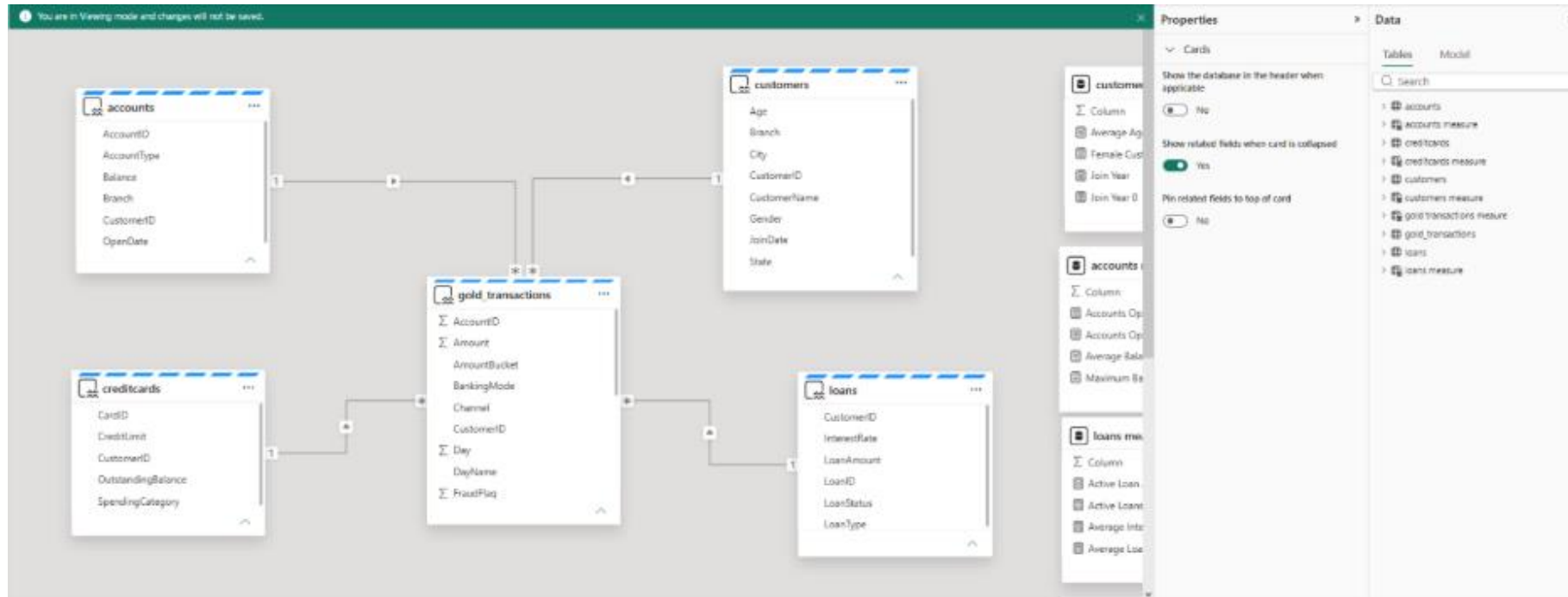
 Run

 Edit

+ Add schedule

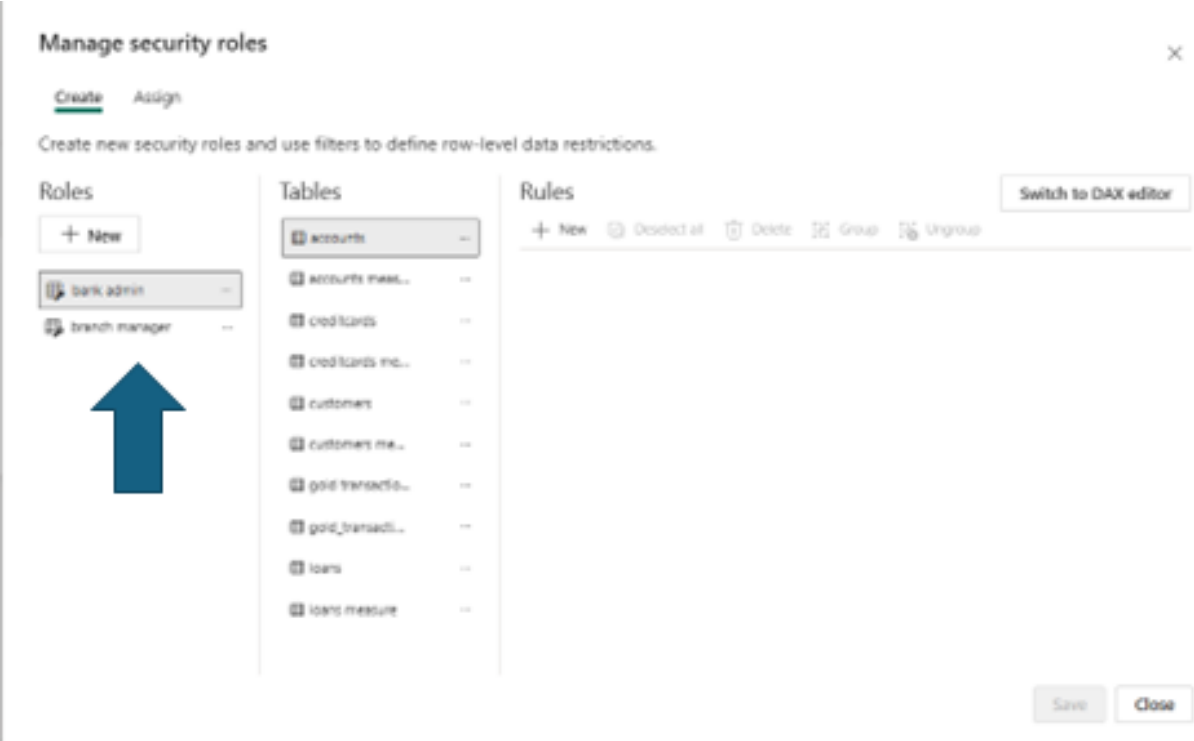
- **Semantic Model and Power BI Reporting**

Purpose: Create a semantic layer and interactive dashboards to enable business users to analyze banking transactions and gain actionable insights.



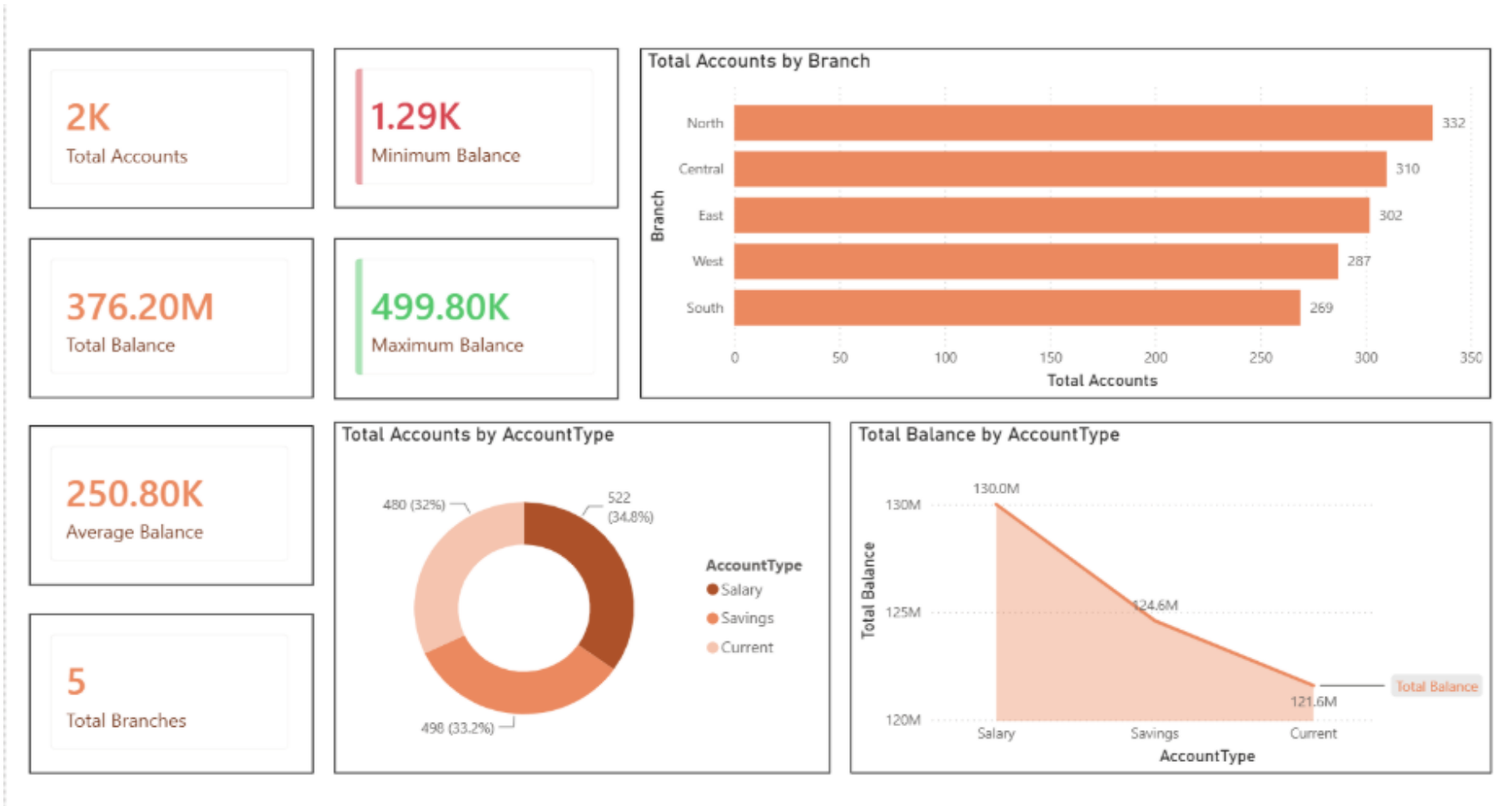
- **Manage Role**

Row-Level Security (RLS) is implemented to restrict data visibility based on user roles, ensuring that users can only view authorized banking data while maintaining security, privacy, and regulatory compliance.



- Monitoring and Business Insights

Purpose: Enable business users to monitor banking operations and make data-driven decisions using real-time insights and dashboards.



24K

Total Transactions

100.40

Smallest Transaction

49.99K

Largest Transaction

589.82M

Total Amount

TransactionID	CustomerID	Amount	TransactionType	TransactionDate
661490	59	38,650.96	TRANSFER	6/19/2026 5:38:34 AM
671058	167	9,977.76	DEPOSIT	6/19/2026 5:38:36 AM
859458	45	29,718.72	BILL PAYMENT	6/19/2026 5:38:38 AM
155299	690	45,881.29	TRANSFER	6/19/2026 5:38:40 AM
921620	730	28,886.99	BILL PAYMENT	6/19/2026 5:38:42 AM

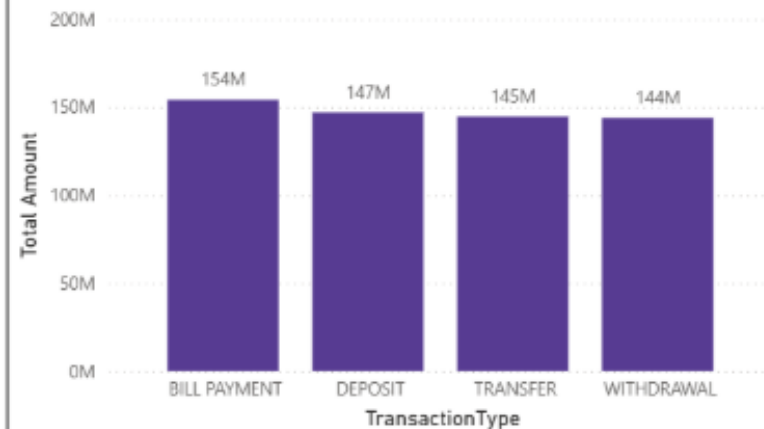
589.82K

Avg Amount Per Custo...

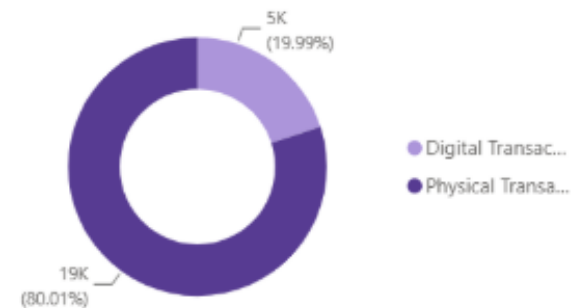
--

Fraud Amount

Total Amount by TransactionType



Digital Transactions and Physical Transactions



Total Amount by MonthName

